Deep Learning compact cheatsheet

STEP 1 Neural network basics

1.1 Neural networks: from perceptrons to MLPs

Neuron $a = \varphi(w \cdot x + b)$. Layer: $h = \varphi(Wx + b)$. Parameters per layer $n_{out}(n_{in} + 1)$.

Perceptron Step output, update $w \mathrel{+}= \eta(y - \hat{y})x$ on mistakes. Converges if separable, $(R/\gamma)^2$ mistakes. Fails on XOR.

MLP Hidden layers with nonlinear activations learn features. Without φ layers collapse to one linear map.

Universal approximation One hidden layer, enough units, any continuous function on a bounded set. Depth is more efficient.

Activations ReLU or GELU hidden. Sigmoid slope ≤ 0.25, saturates. Softmax for classes. Dead ReLU, leaky ReLU.

In practice Scale inputs, MLPClassifier(hidden_layer_sizes, alpha), compare with logistic regression, watch train and test gap.

Neuron $a = \varphi(\mathbf{w} \cdot \mathbf{x} + b)$

Perceptron rule
$\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}, \quad b \leftarrow b + \eta(y - \hat{y})$

Convergence bound $\text{mistakes} \le (R / \gamma)^2$

Layer $\mathbf{h} = \varphi(W\mathbf{x} + \mathbf{b})$

Two layer network
$\hat{y} = \sigma(\mathbf{w}_2 \cdot \tanh(W_1\mathbf{x} + \mathbf{b}_1) + b_2)$

Why nonlinearity $W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x}$

Sigmoid slope $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le 0.25$

Tanh slope $\tanh'(z) = 1 - \tanh^2(z)$

Softmax $p_i = e^{z_i} / \Sigma_j e^{z_j}$

Neuron $a = \varphi(\Sigma_i w_i x_i + b)$

Layer in matrix form $\mathbf{h} = \varphi(W\mathbf{x} + \mathbf{b})$

Number of parameters
$n_{out}(n_{in} + 1)$ per dense layer

Binary cross entropy
$L = -[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$

EXAM On 40 points that a line can separate, the perceptron makes no mistakes after epoch 6. It learns AND in 6 epochs and OR in 4, but after 100 epochs on XOR it still predicts [1, 1, 0, 0] for targets [0, 1, 1, 0]. Minsky and Papert's 1969 book made this limit famous.

EXAM A network with 3 tanh hidden units learns XOR: the loss falls from 0.7471 to 0.0018 in 3000 steps and the outputs become 0, 0.998, 0.998 and 0.003. The hidden values show what happened: each input point got new coordinates in which a single line separates the two classes.

EXAM Sigmoid's slope is 0.25 at 0 and 0.007 at ±5. Tanh's slope is 1 at 0 but 0 at ±5 (to three decimals). ReLU's slope is exactly 1 for every positive input. Multiplying ten sigmoid slopes of at most 0.25 can shrink a gradient by a factor of about one millionth.

EXAM On two noisy half moons, logistic regression reaches 0.861 test accuracy. An MLP with 2 hidden units (9 weights) does no better, 0.856. With 32 units it reaches 0.883, and with two layers of 32 (1185 weights) 0.900, while training accuracy climbs to 0.952. More capacity fits the curve better, but the gap between train and test grows too.

1.2 Backpropagation and automatic differentiation

Backpropagation Chain rule backwards on the graph. Store forward values. Add passes, multiply swaps, $\sigma' = \sigma(1 - \sigma)$.

Accumulation A value used twice gets the sum of both gradients. Zero gradients before each step.

Autodiff Reverse mode: all gradients for about 2 to 3 forward costs. Forward mode: one pass per input.

Matrix gradients $\partial L/\partial W = X^TG$, $\partial L/\partial b$ = row sum of G, $\partial L/\partial X = GW^T$. Softmax CE: $p - y$.

Gradient check Central difference with $h \approx 1e{-}5$, relative error below $1e{-}7$.

Gradient flow Vanishing and exploding from products of slopes. He $2/n_{in}$ for ReLU, Xavier for tanh, clip by norm.

Chain rule $\partial L/\partial x = (\partial L/\partial u)(\partial u/\partial x)$

Add and multiply gates
$u = a + b$: $\partial u/\partial a = 1$. $u = ab$: $\partial u/\partial a = b$

Several paths $\partial L/\partial x = \Sigma_{\text{paths}} \Pi$ local slopes

Reverse mode cost
$\text{cost}(\nabla L) \le c \cdot \text{cost}(L)$, c a small constant

Accumulation
$\partial L/\partial v \mathrel{+}= \partial L/\partial u \cdot \partial u/\partial v$ for every child u of v

Dense layer gradients
$\partial L/\partial W = X^TG, \quad \partial L/\partial \mathbf{b} = \Sigma_{\text{rows}} G, \quad \partial L/\partial X = GW^T$

Softmax with cross entropy $\partial L/\partial \mathbf{z} = \mathbf{p} - \mathbf{y}$

ReLU backward $\partial L/\partial z = \partial L/\partial h \cdot [z > 0]$

Gradient through L layers
$\partial L/\partial \mathbf{h}_1 = \Pi_l (W_l^T \operatorname{diag}(\varphi'(\mathbf{z}_l))) \, \partial L/\partial \mathbf{h}_L$

Xavier initialisation $\text{Var}(W) = 2 / (n_{in} + n_{out})$

He initialisation $\text{Var}(W) = 2 / n_{in}$

Chain rule $dL/dx = (dL/du)(du/dx)$

Multivariable chain rule
$\partial L/\partial x = \Sigma_i (\partial L/\partial u_i)(\partial u_i/\partial x)$

Sigmoid derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Dense layer $\partial L/\partial W = X^TG$, $\partial L/\partial X = GW^T$

EXAM For $f = (x\ y + z)\ w$ followed by a sigmoid and a squared error, the backward pass gives $\partial L/\partial x = 0.0887$ and $\partial L/\partial w = -0.1774$. Nudging x and w by 0.00001 in each direction gives the same numbers to four decimals. This numerical check is how libraries test their gradients.

EXAM In about forty lines, the engine gets $d(x{\cdot}x + x)/dx = 7$ at $x = 3$, adding the three contributions from the three uses of x. It then trains a tanh neuron from a loss of 3.7673 to 0.0047 in 50 steps, and the neuron outputs 0.98 and −0.935 for targets +1 and −1.

EXAM The hand written backward pass agrees with finite differences to a relative error of 7.9e−10 for W1 and 5.6e−10 for W2. Training on 90 overlapping points from 3 classes takes the loss from 1.2295 to 0.3874 and accuracy to 0.822.

EXAM With 20 layers, sigmoid with Xavier initialisation shrinks the gradient from 1.8e−1 at the top to 9.0e−14 at layer 1. Tanh with tiny weights (0.01) is even worse, 6.0e−23. ReLU with Xavier shrinks it about 250 times. ReLU with He initialisation keeps it between 1.1 and 3.3 at every layer.

1.3 Training deep networks: optimisers, normalisation and regularisation

Optimisers SGD $\eta < 2/\lambda_{max}$. Momentum $v = \beta v + g$. RMSprop divides by $\sqrt{s}$. Adam = both plus bias correction.

Defaults AdamW with η about 1e−3 or 3e−4 and decay 0.01 to 0.1. SGD momentum 0.9 for large CNNs.

Schedules Warm up, then cosine or step decay. Batch × k means learning rate × k.

Normalisation BN over the batch per feature, running averages at test. LN per example, for transformers.

Regularisation Weight decay, inverted dropout (divide by p), early stopping on validation, more data and augmentation.

Recipe Check loss ln C at start, overfit one batch, find the rate, then regularise and tune one thing at a time.

SGD $\theta \leftarrow \theta - \eta\, g$

Momentum $v \leftarrow \beta v + g, \quad \theta \leftarrow \theta - \eta v$

RMSprop
$s \leftarrow \rho s + (1 - \rho)g^2, \quad \theta \leftarrow \theta - \eta g / (\sqrt{s} + \varepsilon)$

Adam
$m \leftarrow \beta_1 m + (1 - \beta_1)g, \quad v \leftarrow \beta_2 v + (1 - \beta_2)g^2$

Adam update
$\hat{m} = m/(1 - \beta_1{}^t), \quad \hat{v} = v/(1 - \beta_2{}^t), \quad \theta \leftarrow \theta - \eta\hat{m}/(\sqrt{\hat{v}} + \varepsilon)$

Step decay $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/s \rfloor}$

Cosine decay $\eta_t = \tfrac{1}{2}\eta_0(1 + \cos(\pi t/T))$

Linear warm up $\eta_t = \eta_0 \cdot (t + 1)/w$ for $t < w$

Batch norm
$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B{}^2 + \varepsilon)}, \quad y = \gamma\hat{x} + \beta$

Running mean $\mu \leftarrow 0.9\mu + 0.1\mu_B$

Layer norm
Same formula with μ and σ over one example's features

L2 penalty $L' = L + (\lambda/2)\|\mathbf{w}\|^2$

Weight decay step $\mathbf{w} \leftarrow \mathbf{w}(1 - \eta\lambda) - \eta g$

Inverted dropout
$\tilde{h} = h \cdot m / p$, $m \sim \text{Bernoulli}(p)$

Stability limit for gradient descent
$\eta < 2 / \lambda_{max}$

Condition number $\kappa = \lambda_{max} / \lambda_{min}$

Momentum terminal speed $v \to g / (1 - \beta)$

Adam bias correction $E[m_t] \approx (1 - \beta_1^t)\, E[g]$

EXAM On the valley $f = \frac{1}{2}(x^2 + 100y^2)$, plain SGD is stable only below learning rate 2/100 = 0.02. At its best safe rate 0.019 it reaches a loss of 0.80 after 60 steps, and at 0.021 it explodes to about 10 million. Momentum reaches 0.105 with a rate of only 0.01, Adam reaches 0.103 and RMSprop reaches 0.00005. RMSprop wins so clearly here because this valley lines up with the axes, so scaling each coordinate separately fixes the problem completely. A tilted valley would be harder for it.

EXAM Mini batch SGD with batch 8 on a noisy linear regression ends 0.0167 above the best possible loss with a constant learning rate. Step decay, cosine decay and warm up with cosine all end about 0.0002 above it, roughly 80 times closer, with the same number of steps. For batch size, batch 1 makes 6,000 noisy updates and ends 0.177 above the best, batch 64 makes 90 updates and ends 0.0007 above, and the full batch makes only 6 updates in 6 epochs and is still 2.56 above.

EXAM In a 10 layer tanh network with weights a little too small, the activation spread falls from 0.415 at layer 1 to 0.0007 at layer 10. With batch normalisation before each tanh, it stays at about 0.63 in every layer. After BN with $\gamma = 1$ and $\beta = 0$ each feature has mean 0 and standard deviation 1, and $\gamma = 2$ or $\beta = 10$ rescale or shift it exactly as asked.

EXAM A network with two layers of 64 tanh units, trained on 16 noisy points of sin(x), reaches a training error of 0.0026 but a validation error of 0.596. Weight decay 0.01 gives a validation error of 0.111, dropout with keep 0.8 gives 0.132, and early stopping halts at epoch 447 with 0.118. The noise alone gives about 0.09, so these are close to the best possible.

STEP 2 Architectures

2.1 Convolutional neural networks

Convolution Slide a k × k filter, multiply and add. Weight sharing, local connections. Output $\lfloor (n + 2p - k)/s \rfloor + 1$.

Weights $(k^2 C_{in} + 1) C_{out}$ per conv layer, independent of image size. 1 × 1 conv mixes channels.

Pooling Max or average over windows, halves the size, no weights. Gradient goes to the arg max.

Receptive field $r \leftarrow r + (k - 1)dj$, $j \leftarrow js$. Three 3 × 3 = one 7 × 7 with fewer weights.

Architectures LeNet, AlexNet (ReLU, GPUs), VGG (3 × 3), GoogLeNet (Inception), ResNet (h + F(h)), EfficientNet, ViT.

Practice Pretrained backbone, new head, augmentation that keeps labels true, then fine tune with a smaller learning rate.

Convolution (as used in deep learning)
$Y[i, j] = \Sigma_{a, b}\, X[i \cdot s + a, j \cdot s + b] \cdot K[a, b] + b$

Output size $o = \lfloor (n + 2p - k) / s \rfloor + 1$

Weights in a conv layer
$k \times k \times C_{in} \times C_{out} + C_{out}$

Receptive field $r_l = r_{l-1} + (k_l - 1) \cdot d_l \cdot j_{l-1}$

Jump $j_l = j_{l-1} \cdot s_l$

Max pooling
$Y[i, j] = \max$ over the window of X

Filter gradient
$\partial L / \partial K[a, b] = \Sigma_{i, j}\, \partial L / \partial Y[i, j] \cdot X[i + a, j + b]$

Max pool gradient
Goes to the arg max of each window, 0 elsewhere

Residual block $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l, W_l)$

Gradient through a residual block
$\partial L / \partial \mathbf{h}_l = \partial L / \partial \mathbf{h}_{l+1} (I + \partial F / \partial \mathbf{h}_l)$

Output size $o = \lfloor (n + 2p - k)/s \rfloor + 1$

Conv weights $(k^2 C_{in} + 1)\, C_{out}$

Conv multiply adds $o^2 \cdot k^2 \cdot C_{in} \cdot C_{out}$

Receptive field $r \leftarrow r + (k - 1)\, d\, j$, $j \leftarrow j\, s$

EXAM On an image that is dark on the left and bright on the right, the vertical edge filter gives 27 exactly where the brightness jumps and 0 elsewhere, and the horizontal edge filter gives 0 everywhere. A 224 × 224 image with a 7 × 7 filter, padding 3 and stride 2 gives a 112 × 112 output, the first layer of ResNet.

EXAM 2 × 2 max pooling turns the 4 × 4 map into [[6, 5], [7, 9]]. A 3 × 3 convolution with 16 filters on a 32 × 32 × 3 image has 448 weights, while a dense layer producing the same 32 × 32 × 16 output would need about 50.3 million, 112,384 times more. Through conv, conv, pool, conv, pool, conv the receptive field grows 3, 5, 6, 10, 12, 20.

EXAM On the 8 × 8 scikit learn digits, trained with SGD with momentum for 8 epochs, test accuracy rises from 0.88 after the first epoch to about 0.97. The network has only 1,370 weights: 80 in the 8 filters and 1,290 in the final dense layer.

EXAM With 50 layers and small weights, the gradient that reaches layer 1 of a plain network is 2.7e−50 times the gradient at the output: it has vanished. With the same weights and residual connections it is about 10.5 times the output gradient, because the skip path carries it straight through. Real ResNets also use batch normalisation to keep the sizes steady.

2.2 Recurrent networks, LSTM and GRU

RNN $h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b)$, shared weights, any length. Train with BPTT.

Long term problem Gradient across k steps is a product of k Jacobians: vanishes or explodes. Clip for explosions.

LSTM $c = f \odot c + i \odot g$, $h = o \odot \tanh(c)$. Cell path multiplied only by f. Positive forget bias.

GRU $h = (1 - z)h + z\, \tilde{h}$, reset gate r. 3 blocks against 4 for LSTM.

Weights blocks × $(n_{in} H + H^2 + H)$: RNN 1, GRU 3, LSTM 4.

Variants Bidirectional needs the whole sequence. Encoder decoder with teacher forcing. Attention fixes the bottleneck.

RNN cell $\mathbf{h}_t = \tanh(\mathbf{x}_t W_{xh} + \mathbf{h}_{t-1} W_{hh} + \mathbf{b})$

Output $\mathbf{y}_t = \mathbf{h}_t W_{hy} + \mathbf{c}$

BPTT $\partial L / \partial W_{hh} = \Sigma_t\, \partial L_t / \partial W_{hh}$

Gradient across k steps
$\partial \mathbf{h}_t / \partial \mathbf{h}_{t-k} = \Pi_{j=0}^{k-1} \operatorname{diag}(1 - \mathbf{h}_{t-j}^2)\, W_{hh}^T$

Rough size $\approx (\gamma \cdot \sigma_{max}(W_{hh}))^k$

Gates $\mathbf{i}, \mathbf{f}, \mathbf{o} = \sigma(\mathbf{x}_t W + \mathbf{h}_{t-1} U + \mathbf{b})$

Candidate $\mathbf{g} = \tanh(\mathbf{x}_t W_g + \mathbf{h}_{t-1} U_g + \mathbf{b}_g)$

Cell update $\mathbf{c}_t = \mathbf{f} \odot \mathbf{c}_{t-1} + \mathbf{i} \odot \mathbf{g}$

Hidden state $\mathbf{h}_t = \mathbf{o} \odot \tanh(\mathbf{c}_t)$

Gradient along the cell $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \operatorname{diag}(\mathbf{f})$

GRU update $\mathbf{h}_t = (1 - \mathbf{z}) \odot \mathbf{h}_{t-1} + \mathbf{z} \odot \tilde{\mathbf{h}}_t$

GRU candidate
$\tilde{\mathbf{h}}_t = \tanh(\mathbf{x}_t W + (\mathbf{r} \odot \mathbf{h}_{t-1}) U + \mathbf{b})$

Weights per layer gates × $(n_{in} H + H^2 + H)$

RNN $\mathbf{h}_t = \varphi(\mathbf{x}_t W_{xh} + \mathbf{h}_{t-1} W_{hh} + \mathbf{b})$

LSTM cell
$\mathbf{c}_t = \mathbf{f} \odot \mathbf{c}_{t-1} + \mathbf{i} \odot \mathbf{g}$, $\mathbf{h}_t = \mathbf{o} \odot \tanh(\mathbf{c}_t)$

GRU $\mathbf{h}_t = (1 - \mathbf{z})\mathbf{h}_{t-1} + \mathbf{z}\tilde{\mathbf{h}}_t$

Memory after k steps with forget value f
$c_{t+k} \approx f^k\, c_t$

EXAM A 4 unit RNN reads 5 steps and then 100 steps, and its state is still just 4 numbers. Backpropagating from step 100, the sensitivity of the final state to earlier states falls from 1.39 one step back to 0.61 ten steps back, 0.0088 twenty steps back and 2.0e−7 fifty steps back. We checked the value ten steps back against finite differences.

EXAM The task: see a number at step 1 with a marker, then noise as large as the signal, and output the number at the end. Always answering 0 scores 0.333. A 16 unit plain RNN trained with BPTT and clipping reaches a test error of 0.0003 at 5 steps and 0.0007 at 20 steps, but 0.328 at 40 steps and 0.345 at 60 steps: no better than guessing 0.

EXAM On the same memory task, an LSTM with the usual forget bias of 1 learns 20 steps (error 0.0006) but fails at 60 steps (0.345). With forget bias 5, so that f starts near 0.99, it learns 60 steps with an error of 0.0010, where the plain RNN failed already at 40. The hand written LSTM gradients were checked against finite differences.

EXAM With the update gate bias set to −6, a GRU copies its state almost unchanged through 50 noisy steps: (0.5, −0.5, 0.2, 0.9) becomes (0.37, −0.49, 0.09, 0.81). For 100 inputs and 128 hidden units, an RNN layer has 29,312 weights, a GRU 87,936 and an LSTM 117,248.

2.3 Attention and transformers

Attention $\text{softmax}(QK^T/\sqrt{d_k})V$. Q, K, V are learned maps of the input. Weights sum to 1.

Masking Causal mask for decoders (GPT). Padding masks hide padding tokens.

Block MHA + FFN (4d hidden), each with residual and layer norm. About $12d^2$ weights. $d_{head} = d/h$.

Positions Sinusoidal, learned or rotary encodings. Attention alone is order blind.

Why it won Path length 1, parallel over positions. Cost $O(T^2 d)$, helped by FlashAttention and KV caches.

Models and decoding BERT encoder (masked), GPT decoder (next token), T5 encoder decoder, ViT patches. Temperature, top k, top p, perplexity.

Scaled dot product attention
$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$

Projections
$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$

Causal mask $\text{score}_{ij} = -\infty$ for $j > i$

Multi head attention
$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$

Transformer block (post norm)
$\mathbf{z} = \text{LN}(\mathbf{x} + \text{MHA}(\mathbf{x})), \quad \mathbf{y} = \text{LN}(\mathbf{z} + \text{FFN}(\mathbf{z}))$

Feed forward network
$\text{FFN}(\mathbf{x}) = \max(0, \mathbf{x}W_1 + \mathbf{b}_1)W_2 + \mathbf{b}_2$

Sinusoidal encoding
$PE(pos, 2i) = \sin(pos / 10000^{2i/d}), \quad PE(pos, 2i + 1) = \cos(\ldots)$

Language model $p(\mathbf{x}) = \Pi_t\, p(x_t \mid x_{<t})$

Temperature $p_i = \text{softmax}(z_i / T)$

Perplexity $\text{PPL} = \exp(-(1/N)\, \Sigma \ln p(x_t \mid x_{<t}))$

Attention $\text{softmax}(QK^T/\sqrt{d_k} + M)V$

Shapes
Q: $T \times d_k$, K: $T \times d_k$, V: $T \times d_v$ ⇒ output $T \times d_v$

Heads $d_{\text{head}} = d_{\text{model}} / h$

Cost
$O(T^2 d)$ time, $O(T^2)$ memory for the scores

EXAM With 4 hand picked tokens, the first query puts weight 0.539 on the key that points the same way and 0.065 on the one that points away. A query of all zeros gives equal weights of 0.25, so it simply averages the values. With the causal mask, the first token can only attend to itself (weight 1). For random vectors, the spread of q · k grows from 2.0 at d = 4 to 22.6 at d = 512, and dividing by √d brings it back to 1.

EXAM A block maps 6 vectors of size 16 to 6 vectors of size 16, and each head's attention rows sum to 1. Without positional encodings, reversing the input tokens only reverses the outputs, which shows that attention alone cannot see order. One block with $d_{\text{model}} = 768$ has about 7.1 million weights, so BERT base's 12 blocks hold 85 million. GPT 3's 96 blocks of size 12,288 hold about 174 billion.

EXAM On the exact memory task where the plain RNN failed from 40 steps and the LSTM needed a forget bias of 5 to manage 60, a single attention head learns 20, 60 and even 200 steps in 600 training steps. Its test error is below 1e−5, and it puts 0.987, 0.996 and 1.000 of its attention on the marked token.

EXAM For "The cat sat on the", temperature 0.5 gives "mat" a probability of 0.867, temperature 2 lowers it to 0.456 and spreads the rest. Top k = 2 keeps mat and sofa. Top p = 0.9 keeps mat, sofa and roof, which hold 0.984 of the probability. True next words with probabilities 0.60, 0.25, 0.10 and 0.40 give a perplexity of 3.59. Going from 2,048 to 8,192 tokens multiplies the attention scores per head by 16.

STEP 3 Generative models

3.1 Autoencoders and variational autoencoders

Autoencoder Encoder to a small code, decoder back. Train on reconstruction error, no labels.
Versus PCA Linear autoencoder = PCA subspace. Nonlinear layers rebuild curved data better.
Uses Denoising (noisy in, clean out). Anomalies: train on normal data, flag high reconstruction error.
VAE Encoder gives μ, log σ². $z = \mu + \sigma\varepsilon$. Loss = reconstruction + KL = negative ELBO.
KL $\frac{1}{2}\Sigma(\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$, zero only for N(0, 1). β weights it.
Discrete and latent VQ-VAE: nearest codebook index, $\log_2$(size) bits. Latent spaces make diffusion cheap.

Autoencoder $\mathbf{z} = f_{\text{enc}}(\mathbf{x}), \quad \hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z})$

Reconstruction loss
$L = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$, or binary cross entropy for values in [0, 1]

Linear autoencoder
Optimal solution spans the top k principal components

Reparameterisation
$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim N(0, I)$

ELBO
$\ln p(\mathbf{x}) \geq E_{q(z|x)}[\ln p(\mathbf{x}|z)] - KL(q(z|\mathbf{x}) \,\|\, p(z))$

Gaussian KL
$KL = \frac{1}{2}\, \Sigma_j\, (\mu_j^2 + \sigma_j^2 - 1 - \ln \sigma_j^2)$

Quantisation $z_q = e_k, \quad k = \text{argmin}_j \|z_e - e_j\|$

Bits per token $\log_2$(codebook size)

Reconstruction (Gaussian)
$-\ln p(\mathbf{x}|z) \propto \|\mathbf{x} - \hat{\mathbf{x}}\|^2$

Reconstruction (Bernoulli)
$-\Sigma\, [x \ln \hat{x} + (1 - x) \ln(1 - \hat{x})]$

KL of one dimension $\frac{1}{2}(\mu^2 + \sigma^2 - 1 - \ln \sigma^2)$

Log variance $\sigma = \exp(\frac{1}{2} \log \sigma^2)$

EXAM Compressing 8 × 8 digits (64 pixels) to 2 numbers, PCA reaches a test error of 0.0534 and a nonlinear autoencoder 0.0357. With 8 numbers, PCA gives 0.0245 and the autoencoder 0.0147. Just predicting the average image gives 0.0743.

EXAM Adding noise with standard deviation 0.3 gives the test digits an error of 0.0524, and the denoising network brings it down to 0.0324. An autoencoder trained only on digits 0 to 4 rebuilds them with error 0.0322 but digits 5 to 9 with 0.0605, and using the error as an anomaly score separates the two groups with a ROC AUC of 0.901.

EXAM A VAE with a 2 number code trained on the digits for 200 epochs lowers its loss (negative ELBO) from 36.3 to 24.4 nats per image: reconstruction 21.75 plus KL 2.62. Different digits get different average codes, for example (−1.33, −0.39) for 1 and (1.33, 0.92) for 0. Decoding the point halfway between them gives a shape in between, a smooth path through the code space.

EXAM Snapping 8 number codes of the digits to a codebook of 4 vectors (2 bits per image) gives an error of 0.0549, 16 vectors (4 bits) give 0.0375 and 256 vectors (8 bits) give 0.0273, approaching the continuous code's 0.0239. Each image becomes a single token id. Here the codebook is learned with k means on PCA codes, a simple stand in for a trained VQ-VAE.

3.2 Generative adversarial networks

GAN game Generator: noise to fake. Discriminator: real or fake. Alternate updates, min over G of max over D.
Losses $L_D = -\ln D(x) - \ln(1 - D(G(z)))$. Generator: $-\ln D(G(z))$ (non saturating).
Theory $D^* = p_{\text{data}} / (p_{\text{data}} + p_g)$. $V(D^*, G) = -\ln 4 + 2$ JS. Optimum $p_g = p_{\text{data}}$, $D = \frac{1}{2}$.
Failures Saturation, mode collapse, oscillation, D too strong. Fixes: minibatch features, TTUR, penalties, augmentation.
WGAN Critic outputs real numbers, estimates earth mover distance. Keep 1 Lipschitz: gradient penalty or spectral norm.
Variants and FID DCGAN, conditional, pix2pix, CycleGAN (cycle loss), StyleGAN. FID: feature means and covariances, lower is better.

Minimax game
$\min_G \max_D V(D, G) = E_{x \sim p_{\text{data}}}[\ln D(x)] + E_{z \sim p_z}[\ln(1 - D(G(z)))]$

Discriminator loss
$L_D = -\ln D(x) - \ln(1 - D(G(z)))$

Generator loss (non saturating)
$L_G = -\ln D(G(z))$

Optimal discriminator
$D^*(x) = p_{\text{data}}(x) / (p_{\text{data}}(x) + p_g(x))$

Value at the optimum
$V(D^*, G) = -\ln 4 + 2\, \text{JS}(p_{\text{data}} \,\|\, p_g)$

Gradient on the logit t
saturating: $-D$, non saturating: $-(1 - D)$

Wasserstein distance
$W(p, q) = \inf_\gamma E_{(x, y) \sim \gamma}\|x - y\|$

Kantorovich Rubinstein duality
$W(p, q) = \sup_{\|f\|_L \leq 1} E_p[f(x)] - E_q[f(x)]$

Gradient penalty
$\lambda\, E[(\|\nabla_{\hat{x}} f(\hat{x})\| - 1)^2], \quad \lambda = 10$

Fréchet distance of Gaussians
$FID = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$

Cycle consistency
$L_{\text{cyc}} = \|F(G(x)) - x\|_1 + \|G(F(y)) - y\|_1$

Transposed convolution size
out = (in − 1) × stride − 2 × pad + kernel

GAN value
$V(D, G) = E_{\text{data}}[\ln D(x)] + E_z[\ln(1 - D(G(z)))]$

Optimal D $D^*(x) = p_{data} / (p_{data} + p_g)$

JS divergence
$JS(p \| q) = ½ KL(p \| m) + ½ KL(q \| m), \quad m = ½(p + q)$

WGAN critic
$\max_{f \text{ 1 Lipschitz}} E_{data}[f(x)] - E_z[f(G(z))]$

EXAM In this tiny GAN the real numbers come from a bell curve with mean 4.00 and spread 0.50. With the original minimax loss the generator is still at mean 1.15 after 100 steps and its spread later collapses to 0.00. The non saturating loss reaches mean 3.83 by step 100 and settles at mean 4.03, but with a spread of only 0.17. When the discriminator can also see the spread of each batch, the generator ends at mean 4.00 and spread 0.50.

EXAM With the real data N(0, 1) and the generator N(m, 1), the best discriminator scores x = 0 as 0.500 when m = 0, 0.622 when m = 1 and 0.989 when m = 3. The GAN value V(D*, G) matches −ln 4 + 2 JS exactly: −1.3863 at m = 0, −1.1635 at m = 1, −0.3327 at m = 3 and 0.0000 at m = 10, where JS reaches its ceiling ln 2 = 0.6931.

EXAM Shifting a uniform distribution on [0, 1] to the right, the JS divergence is 0.345 at a shift of 0.5 and then stuck at 0.693 for every shift of 1 or more, once the two stop overlapping. The Wasserstein distance keeps growing with the shift: 0.497, 0.994, 1.996, 4.997 and 9.996.

EXAM Using PCA features of the digits in place of Inception features, other real digits score 0.03 against the test digits. Adding noise with spread 0.2 gives 0.10 and spread 0.4 gives 0.33. A set of only real 1s scores 3.23, even though every image in it is a real digit: FID punishes mode collapse because the feature covariance is wrong. The average image repeated scores 4.04.